ProctoEase: Comprehensive System Requirements Specification and Implementation Roadmap for an AI-Assisted Online Examination Platform

Executive Overview and System Paradigm

The fundamental mechanisms through which educational institutions and corporate enterprises evaluate human capability have undergone an irreversible paradigm shift. The global disruption caused by the rapid transition to remote learning and digital recruitment exposed critical vulnerabilities in traditional examination methodologies.[1] As academia and industry scrambled to maintain operational continuity, monolithic enterprise proctoring solutions initially dominated the market.[1] However, a rigorous analysis of the current technological landscape reveals that these legacy systems are increasingly misaligned with the needs of modern organizations. They are prohibitively expensive for small-to-medium enterprises (SMEs), technically complex, environmentally detrimental due to their reliance on continuous high-definition video streaming, and often function as opaque algorithmic arbiters that erode user trust.[1] The severity of the problem is illustrated by large-scale failures; for example, the LSAT exam was reverted to in-person only after organized cheating bypassed its online version.[1]

In response to these systemic deficiencies, a new architectural paradigm has emerged. This document serves as an exhaustive Software Requirements Specification (SRS) and implementation roadmap for ProctoEase, a next-generation online examination and proctoring system. By synthesizing insights from academic literature, system architecture frameworks, advanced algorithmic plagiarism detection models, and global data protection mandates, this report provides a definitive blueprint for an equitable, transparent, and ecologically responsible digital assessment platform. The proposed system avoids the pitfalls of "Big Brother" surveillance by utilizing Edge Artificial Intelligence (AI) purely as an assistive diagnostic tool within a Human-in-the-Loop architecture, ensuring that final determinations of academic integrity are made by human evaluators using explainable evidence.[1]

Core System Architecture and Integration Strategy

The architectural foundation of the ProctoEase platform is highly modular, asynchronous, and capable of handling immense concurrent loads during high-volume testing events without degrading user experience. The system transitions away from traditional synchronous monolithic designs in favor of a decoupled, microservices-oriented topology utilizing

state-of-the-art open-source frameworks.[1]

Frontend Architecture: React and Browser APIs

The client-side application is engineered using React, providing a highly responsive Single Page Application (SPA) experience. The user interface is bifurcated to handle both the candidate's localized examination experience and the administrator's analytical review dashboard. For technical and coding assessments, the frontend integrates the Monaco Editor, which is the foundational technology behind Microsoft's Visual Studio Code.[1] The integration of the Monaco Editor brings desktop-grade text editing features directly to the browser, including intelligent syntax highlighting, multi-cursor support, and code folding.[1] This design choice drastically reduces the cognitive friction typically associated with browser-based text areas, allowing candidates to operate within an ergonomic professional development environment.[1]

To facilitate continuous localized proctoring without severe bandwidth consumption, the React application leverages modern HTML5 Browser APIs. The MediaDevices API captures webcam and microphone streams exclusively for localized algorithmic processing and intermittent snapshot extraction, avoiding continuous video uploads.[1] The Page Visibility API monitors document focus, triggering immediate telemetry events whenever a candidate switches tabs or minimizes the secure browser context.[1] Concurrently, the Web Audio API analyzes ambient noise for distinct human speech patterns locally, transmitting only mathematical anomaly scores rather than raw, privacy-invasive audio feeds to the server.[1]

Backend Orchestration: FastAPI

The backend infrastructure serves as the central nervous system of the assessment canvas. The strategic selection of FastAPI as the core backend framework represents a significant departure from legacy synchronous systems like Django or Flask.[1] Built upon Starlette for ASGI web routing and Pydantic for strict data validation, FastAPI provides native asynchronous capabilities.[1] This architecture allows the server to manage thousands of concurrent WebSocket connections and RESTful requests without the performance bottlenecks inherent in thread-blocking operations.[1]

Recent performance benchmarks validate this architectural choice. FastAPI can handle upwards of 21,000 requests per second with a latency of merely two milliseconds under heavy concurrent load, whereas Node.js handles similar loads at five milliseconds, and Django's synchronous WSGI architecture struggles with fifteen-millisecond latencies.[3] For an AI-assisted proctoring system that must ingest thousands of simultaneous telemetry data points, the non-blocking event loops of FastAPI ensure real-time responsiveness and platform stability.[2]

Asynchronous Task Processing: Celery and Redis

To maintain low latency on the primary FastAPI nodes during computationally intensive operations, the architecture implements a distributed task queue managed by Celery, utilizing Redis as the highly performant in-memory message broker.[1] Operations such as calculating complex structural plagiarism matrices across thousands of submissions, processing image snapshots for administrative review, and aggregating final heuristic risk scores are completely decoupled from the main request-response cycle.[1]

When a candidate submits an examination or a proctoring anomaly is registered, the FastAPI server immediately returns a success acknowledgment to the client while simultaneously pushing the evaluation payload to the Redis queue.[4] Background Celery worker nodes retrieve these jobs from the queue and process the data asynchronously. To guarantee production-grade reliability, the Celery architecture employs horizontal scaling by spinning up multiple worker nodes based on queue depth, alongside strict task idempotency design, ensuring that tasks can be safely retried in the event of distributed system failures without corrupting the database state.[4]

Sandboxed Code Execution Engine: Judge0

For technical recruitment and computer science education, the ability to practically evaluate coding proficiency is paramount. The platform integrates Judge0, a highly robust, fast, and scalable open-source online code execution system.[1] Executing arbitrary, untrusted candidate code on host servers presents an existential security risk, exposing the infrastructure to malicious payloads, infinite loops, and resource exhaustion attacks.[1] Judge0 mitigates this risk by operating as an entirely isolated, sandboxed execution environment.[1]

The FastAPI backend transmits the candidate's raw source code, the target compiler identifier, and hidden test cases via an HTTP REST API to the Judge0 instance.[1] Utilizing Linux namespaces and cgroups, Judge0 enforces draconian limitations on the execution context, preventing the code from accessing the host file system or network.[1]

Judge0 API Parameter	Operational Function within Sandbox	Security and Evaluation Impact
source_code	Receives the raw text of the candidate's program.	The primary artifact evaluated for syntax, execution, and logic.
language_id	Integer specifying the target compiler/interpreter.	Ensures correct execution context across over sixty

		supported programming languages.
cpu_time_limit	Hard limit on execution time (e.g., 2 seconds).	Prevents infinite loops and malicious denial-of-service (DoS) attempts against the sandbox.
memory_limit	Maximum address space allocated in kilobytes.	Prevents memory leak vulnerabilities and enforces memory-efficient algorithmic logic.
expected_output	The hidden test case solution matrix.	Facilitates the automated, objective grading of candidate logic against predefined edge cases.

Upon completion or termination of the sandboxed process, the engine returns highly detailed telemetry, including exact compilation errors, standard output streams, execution times, and kernel exit signals.[1] This data is asynchronously retrieved by the Celery workers and seamlessly integrated into the candidate's final evaluation profile, allowing organizations to evaluate real-world software engineering capabilities rather than trivial algorithm memorization.[1]

Edge Artificial Intelligence and Algorithmic Proctoring Mechanisms

Traditional AI proctoring solutions rely on continuous, high-definition video streaming to centralized cloud servers for remote algorithmic analysis. This approach is highly bandwidth-intensive, computationally expensive, environmentally degrading, and raises severe privacy concerns.[1] ProctoEase completely re-architects this monitoring workflow through Edge AI processing and transparent heuristic modeling.

Client-Side Analytics via MediaPipe and face-api.js

The platform operates its computer vision and behavioral analysis models entirely within the candidate's secure browser environment using lightweight, highly optimized machine learning frameworks such as MediaPipe and TensorFlow.js (face-api.js).[1] By executing facial landmark detection, gaze tracking, and object detection locally via WebAssembly, the platform

circumvents the necessity of transmitting heavy video feeds.[1] The system records events and extracts periodic photographic snapshots only when specific violations are detected, drastically reducing network payload sizes and server-side compute requirements.[1]

The system relies on a multi-modal detection strategy utilizing several distinct neural network architectures:

- **Identity Verification and Liveness Detection:** A convolutional neural network (CNN) pipeline continuously processes webcam frames to detect the primary face and extract distinct facial landmarks. This ensures the candidate matches the pre-enrolled baseline profile and prevents spoofing via static photographs or recorded videos using liveness detection networks.[1]
- **Multiple-Person Detection:** The vision model continuously scans the background matrix to verify that only one distinct human entity is present within the testing environment. If an unauthorized individual enters the frame, the system immediately logs the anomaly.[1]
- **Head Pose and Gaze Estimation:** Utilizing a solvePnP algorithm on the detected facial landmarks, the system calculates the three-dimensional orientation of the candidate's head.[1] This allows the platform to identify prolonged periods where the candidate's gaze deviates significantly from the screen, which may indicate the consultation of physical notes or hidden secondary devices.[1]
- **Object Detection:** Pretrained YOLO (You Only Look Once) network variants scan the localized frames for prohibited objects, specifically identifying the presence of mobile phones, smartwatches, or textbooks within the candidate's immediate operational area.[1]

Heuristic Risk Scoring Model

A primary critique of traditional AI proctoring systems is their reliance on "black-box" models that generate unexplainable, biased cheating probabilities. Empirical research demonstrates that autonomous AI proctoring suffers from unacceptably high false-positive rates, frequently misinterpreting benign movements as academic dishonesty and disproportionately flagging demographic minorities.[1] To resolve this, the platform utilizes a transparent, deterministic heuristic model to calculate a specific "Proctor Risk Score" ranging from zero to one hundred.[1]

Each specific behavioral anomaly detected by the local AI models or browser APIs is assigned a configurable mathematical weight.[1]

Behavioral Anomaly	Telemetry Source Mechanism	Suggested Heuristic Weight	Interpretation and Rationale
Window Blur / Tab Switch	Browser Page Visibility API	High (+20)	Indicates the candidate navigated away

			from the secure browser context, likely to search for unauthorized external solutions.
Multiple Faces Detected	MediaPipe / OpenCV Model	Critical (+35)	Strong evidence of physical collusion or unauthorized personnel providing assistance within the testing environment.
Face Missing from Frame	MediaPipe / OpenCV Model	Medium (+15)	Suggests the candidate may be consulting physical notes below desk level or using an obscured secondary device.
Ambient Audio Spike	Web Audio API Analysis	Low (+5)	May indicate unauthorized communication, but is highly susceptible to benign environmental noise (e.g., traffic or sirens).
Gaze Deviation	solvePnP Pose Estimation	Moderate (+10)	Indicates the candidate is continuously looking off-screen, warranting a review of the room's physical layout.

As the examination progresses, the risk engine aggregates these weighted events. If the

cumulative score crosses predefined organizational thresholds, the session is locked or flagged for human administrative review.[1]

Explainable AI (XAI) and the Interactive Evidence Timeline

Because algorithmic bias and false positives remain an inherent risk in computer vision, the platform explicitly rejects autonomous punitive actions. The architecture mandates a "Human-in-the-Loop" workflow where the AI serves purely as an assistive diagnostic telemetry tool.[1] This philosophy is realized through the integration of Explainable AI (XAI) design patterns.[1]

Instead of presenting an administrator with an arbitrary aggregate score, the analytical dashboard features an Interactive Evidence Timeline.[1] This visual component maps the entire temporal duration of the assessment along a horizontal axis, plotting every flagged telemetry event as a distinct, color-coded node based on severity.[1] The timeline allows human reviewers to instantly identify behavioral patterns; for instance, a single audio spike might be dismissed as an environmental anomaly, but a cluster of tab switches corresponding precisely with rapid code generation presents a compelling, undeniable narrative of academic misconduct.[1]

Crucially, interacting with any node upon the timeline reveals a captured photographic snapshot of the exact moment the anomaly occurred, alongside a natural language explanation generated by the system (e.g., "At 14:02:15, a secondary face was detected in the upper right quadrant of the frame").[1] Advanced implementations also incorporate scatter plots visualizing the candidate's predicted gaze directions throughout the session.[1] By providing irrefutable, time-stamped visual evidence rather than an abstract probability metric, the final determination of academic integrity remains entirely in the hands of the human evaluator.[1]

Advanced Structural Plagiarism Detection Framework

In technical coding assessments, ensuring the originality of source code is as critical as verifying its logical correctness. Superficial string-matching algorithms are easily defeated by candidates utilizing basic obfuscation techniques, such as renaming variables, inserting arbitrary whitespace, or reordering independent functions.[1] Furthermore, the rapid proliferation of Generative AI and Large Language Models (LLMs) requires highly sophisticated detection mechanisms capable of identifying structural logic theft even when the surface syntax has been heavily modified.[1] The ProctoEase architecture leverages a highly scalable dual-engine approach combining JPlag tokenization and MinHash algorithms.

Structural Logic Analysis via JPlag

JPlag is a robust, academically proven code similarity analyzer that fundamentally discards the superficial textual representation of a program.[1] Upon submission, the JPlag engine

parses the candidate's source code into an Abstract Syntax Tree (AST) and converts the logical flow into a canonical sequence of language-independent tokens.[1] This tokenization process strips away all identifiers, comments, formatting, and specific syntactic sugar, reducing the program to its core structural logic.[1]

JPlag subsequently employs a Greedy String Tiling algorithm to identify the longest contiguous sequences of matching tokens between any two submissions in the database.[1] This method is highly resilient against Type I (exact copy) and Type II (renamed identifiers and changed data types) plagiarism attempts.[1] An essential capability of JPlag for educational settings is template removal. Instructors routinely provide boilerplate starter code for complex projects; JPlag allows administrators to upload this template, which the algorithm automatically isolates and ignores during the similarity comparison phase, preventing false-positive plagiarism flags on universally shared structural elements.[1]

Locality-Sensitive Hashing (LSH) via MinHash

While JPlag is highly precise for structural comparison, the computational complexity of pairwise string tiling becomes a severe processing bottleneck when analyzing massive cohorts involving tens of thousands of submissions.[1] To achieve enterprise-grade scalability, the platform integrates Locality-Sensitive Hashing (LSH) utilizing the MinHash algorithm.[1]

MinHash converts the tokenized Abstract Syntax Tree into mathematical fingerprints or signatures.[1] Unlike traditional cryptographic hashing functions (such as SHA-256), where even a microscopic character change alters the entire output hash radically to prevent collisions, LSH is explicitly designed so that structurally similar inputs yield highly similar hash values.[1] By calculating the Jaccard similarity coefficient between these mathematical signatures, the backend can rapidly group suspicious submissions into identical hash buckets.[1]

This methodology allows the system to filter a massive, unwieldy dataset down to a small cluster of highly suspicious submission pairs in a fraction of the time required by traditional methods.[1] Once clustered by MinHash, this refined subset of candidates is passed to the JPlag engine for deep, granular structural comparison and visual evidence generation.[1]

Normalization Against Generative AI Obfuscation

To counteract source code generated or heavily refactored by LLMs, the plagiarism matrix includes an advanced pre-processing normalization phase. Before the tokenization step occurs, the system analyzes the semantic dependencies of the code, removing dead nodes, resolving inline variable expansions, and utilizing topological sorting to revert reordered independent code blocks back to a standardized canonical graph.[1] This virtual de-obfuscation process uncovers the fundamental structural overlaps even when a candidate has utilized an AI agent to heavily mask copied logic, ensuring that the integrity of the assessment scales

securely alongside advancements in generative technologies.[1]

Multi-Tenant Database Design and Row-Level Security

As a Software-as-a-Service (SaaS) platform, ProctoEase must simultaneously serve multiple distinct institutions, corporate enterprises, and educational bodies from a single unified deployment infrastructure. Maintaining strict, verifiable data isolation between these tenants is a paramount security and operational requirement; cross-tenant data leakage is catastrophic to platform trust and legal compliance.[1]

Architectural Rationale: PostgreSQL over MongoDB

A rigorous tool comparison dictates the absolute selection of PostgreSQL over NoSQL alternatives like MongoDB for this specific multi-tenant architecture. While MongoDB offers significant schema flexibility and excels at rapid unstructured writes, SaaS assessment platforms require complex relational aggregations (e.g., calculating average cohort scores against historical baseline metrics across specific departments), strict ACID transactional guarantees, and rigorous data consistency.[11] Modern PostgreSQL provides elite performance for JSONB data types, allowing the platform to store unstructured proctoring telemetry logs flexibly while maintaining a rigid relational structure for users, roles, and examination configurations.[12]

The Pool Model and Row-Level Security (RLS) Implementation

The architecture implements a "Pool Model" for multi-tenancy. Rather than provisioning a completely separate database instance for every single client (the Silo Model, which is cost-prohibitive to scale) or managing thousands of separate database schemas (the Bridge Model, which creates immense migration and maintenance complexity), all tenant data resides within the same shared tables.[10] Every data record across the entire system contains a mandatory tenant_id foreign key column, cryptographically binding that row to a specific organization.[1]

However, relying solely on application-level logic to append WHERE tenant_id =? to every database query introduces an unacceptable risk of human error leading to cross-tenant data exposure.[10] To enforce absolute data isolation at the lowest possible layer, the platform utilizes PostgreSQL's native Row-Level Security (RLS) feature.[10]

By executing administrative commands upon table creation, the database engine intercepts all queries before execution: ALTER TABLE assessments ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation_policy ON assessments FOR ALL USING (tenant_id = current_setting('app.current_tenant_id')::UUID);.[10]

When the FastAPI backend receives an authenticated HTTP request, it extracts the tenant_id from the secure JSON Web Token (JWT). Upon checking out a database connection from the

asynchronous pool (managed via asyncpg), the backend explicitly sets the session variable app.current_tenant_id.[10] From that exact moment, the PostgreSQL engine automatically filters out all rows belonging to other tenants at the disk-read level.[10] This guarantees mathematically verifiable tenant isolation regardless of potential bugs in the Python application logic.[10] To ensure this architecture remains highly performant, composite database indexes are systematically applied to all partitioned tables to facilitate rapid index scans during query execution.[15]

Multi-Tenant Model	Storage Strategy	Isolation Security	Cost Efficiency	Suitability for ProctoEase
Silo Model	Separate Database Instance per Tenant	Maximum (Physical Isolation)	Very Poor	Rejected due to extreme infrastructure scaling costs.
Bridge Model	Separate Schema per Tenant	High (Logical Isolation)	Moderate	Rejected due to schema migration complexity at scale.
Pool Model + RLS	Shared Tables with tenant_id	Very High (Engine-Level Enforcement)	Excellent	**Selected.** Provides optimal scalability with strict, database-enforced security boundaries.

Offline-First Resilience Strategy

A critical vulnerability of traditional cloud-based examination systems is their absolute reliance on an uninterrupted, high-speed broadband connection. In regions characterized by digital inequity, fluctuating bandwidth, or rolling power outages, minor network drops frequently result in catastrophic data loss, candidate panic, and compromised assessments.[1] The ProctoEase architecture elegantly resolves this through an "offline-first" engineering approach leveraging advanced Progressive Web App (PWA) browser capabilities.

Application Shell Caching via Service Workers

Upon initial load, the React frontend registers a Service Worker, a background JavaScript script that operates entirely independent of the main browser thread and acts as a programmable network proxy.[17] During the installation phase, the Service Worker aggressively caches the entire "Application Shell," which includes all HTML, CSS, JavaScript bundles, and static visual assets required to render the interface.[18] Consequently, if the network connection fails mid-assessment, the browser does not default to a catastrophic system error screen. The user interface remains completely functional, rendered flawlessly from the persistent local cache.[1]

Client-Side Persistence via IndexedDB

While the Service Worker manages static assets, dynamic transactional data—such as candidate keystrokes, multiple-choice selections, code snippets, and localized AI proctoring telemetry logs—requires structured, persistent storage.[1] The architecture utilizes the IndexedDB API, an asynchronous, transactional, NoSQL object store built directly into modern web browsers.[19]

When an offline state is detected by the client application, it intercepts outgoing REST API requests. Instead of failing, it systematically writes the payload data to the local IndexedDB instance, tagging each record with a specific synced: false flag.[18] The user interface gracefully displays a subtle, non-intrusive network indicator informing the candidate that they are operating offline but that their examination progress is locally secured, effectively mitigating test-induced anxiety.[1]

Automated Background Synchronization

The system utilizes the Background Synchronization API to guarantee eventual data consistency without requiring manual user intervention.[20] When data is cached locally during an outage, the application registers a sync tag (e.g., sync-exam-payloads) with the SyncManager interface of the Service Worker.[20] As soon as the browser detects the restoration of a stable network connection, it fires a sync event within the Service Worker context, even if the user has navigated away from the application tab.[20]

The worker sequentially reads all unsynchronized records from IndexedDB, batches them into efficient payloads, and dispatches them to the FastAPI backend.[20] Upon successful cryptographic acknowledgment from the server, the local IndexedDB records are permanently purged or updated to synced: true, ensuring zero data loss and maintaining the absolute integrity of the examination process despite localized infrastructure failures.[21]

Security Stack, Cryptography, and Access Control

Security and authorization are integrated deeply into both the network layer and the application logic, ensuring that sensitive assessment data, biometric telemetry, and proprietary organizational configurations remain highly protected against unauthorized access or exploitation.

Identity and Authentication: JWT and OAuth2

The platform utilizes JSON Web Tokens (JWT) implemented via FastAPI's native OAuth2 password bearer dependencies to manage stateless authentication.[22] Upon successful authentication, the server issues a short-lived access token and a securely rotated refresh token.[22] The JWT payload contains essential claims, including the user's unique identifier, their assigned role, and their specific tenant_id.[22] To mitigate Cross-Site Scripting (XSS) attacks, tokens are stored strictly in HttpOnly, Secure, and SameSite=Strict browser cookies rather than vulnerable browser LocalStorage environments.[22] All API traffic is strictly enforced over TLS 1.3 (HTTPS), ensuring data is encrypted in transit, while database volumes utilize AES-256 encryption at rest.[1]

Tenant-Scoped Role-Based Access Control (RBAC)

Within a multi-tenant environment, authorization must be highly granular. The system implements a robust Role-Based Access Control (RBAC) matrix that is strictly scoped to the tenant level.[24] A user holding an "Admin" or "Recruiter" role possesses ultimate authority over their specific organization's data but has absolute zero access visibility into the overarching platform administration or neighboring tenant environments.[26]

FastAPI enforces these roles through custom dependency injection. Route handlers require an injected function (e.g., Depends(require_role("recruiter"))) which explicitly decodes the JWT and validates both the role claim and the tenant_id match before any underlying business logic or database queries are executed, preventing horizontal privilege escalation attacks.[22]

Ethical Compliance, Privacy, and DPDP Act Integration

The integration of biometric AI within educational and recruitment spheres introduces profound ethical complexities and legal liabilities. Surveillance architectures inherently risk violating privacy rights if not strictly governed. ProctoEase is engineered from the ground up to comply strictly with global data protection frameworks, specifically aligning with the rigorous mandates of the India Digital Personal Data Protection (DPDP) Act of 2023.[29]

Minors' Data Protection and Verifiable Consent

Under Section 9 of the DPDP Act, children (defined unequivocally as individuals under the age of 18) are afforded specific categorical protections.[30] The platform integrates mandatory age-gating and Verifiable Parental Consent mechanisms directly into the onboarding flow

prior to any data collection.[32] Furthermore, the system strictly disables any form of behavioral tracking, automated profiling, or targeted advertising for users identified as minors, a direct and non-negotiable mandate of the regulatory framework.[32]

Data Minimization and Retention Policies

Surveillance architectures inherently risk violating data minimization principles. By utilizing snapshot-based Edge AI instead of continuous video streaming, the platform inherently minimizes the volume of sensitive biometric data transferred to and stored on the centralized servers, adhering to the principle of collecting only what is strictly necessary for academic integrity.[1]

To comply with the DPDP Act's Rule 6 and Schedule VI, the system enforces automated retention and erasure policies.[32]

Data Category	DPDP Act Retention Mandate	ProctoEase Automated Policy
Operational Telemetry and Audit Logs	Minimum 1 Year (Schedule VI)	Retained securely for 365 days to support lawful investigations and dispute resolution, then automatically purged.[29]
Personal Data and Biometric Snapshots	Erase when purpose is fulfilled (Rule 6)	Erased immediately upon the finalization of the examination grade and the expiration of the academic challenge period.[32]
Consent Records	Retain as proof of compliance	Immutable cryptographic logs maintained for the duration of the user account lifecycle.[34]

The frontend provides a transparent privacy dashboard, allowing users (and their verified guardians) to easily execute their fundamental rights to access, correct, or definitively erase their personal data at any time, withdrawing consent seamlessly.[33] Furthermore, to address documented concerns regarding automated proctoring software disproportionately flagging individuals based on skin tone or gender, the AI models undergo continuous bias auditing against diverse demographic datasets, with the absolute requirement of human-in-the-loop

review mitigating the risk of systemic algorithmic discrimination.[1]

Societal Impact and UN SDG Alignment

The architectural choice to deprecate continuous video recording in favor of localized browser processing yields a profound societal and environmental impact, directly supporting the United Nations Sustainable Development Goals (SDGs).[1]

- **SDG 13: Climate Action:** Traditional continuous video proctoring transmits gigabytes of data per hour, resulting in massive data center compute strain. Studies indicate that one hour of continuous video streaming can emit between 150g to 1000g of CO_2.[1] By processing telemetry locally and transmitting only lightweight metadata arrays and specific anomaly snapshots, the ProctoEase platform reduces bandwidth consumption by 90% and slashes digital carbon emissions by up to 96% per user session, providing a highly verifiable intervention against the expanding carbon footprint of the ICT sector.[1]
- **SDG 4: Quality Education & SDG 10: Reduced Inequalities:** Legacy assessment systems demand high-speed, persistent broadband, effectively disenfranchising students in rural, developing, or socio-economically disadvantaged regions.[1] The offline-first architecture bridges this digital divide. By ensuring that candidates in areas with unreliable grid infrastructure are not excluded from digital education and certification opportunities, the platform democratizes access to secure, verifiable assessment capabilities regardless of local infrastructure quality.[1]

Deployment Orchestration and CI/CD Pipeline

To ensure high availability, rapid feature iteration, and operational stability, the platform utilizes modern containerization technologies and automated continuous integration/continuous deployment (CI/CD) pipelines.[1]

Containerization Strategy via Docker

The entire technology stack is containerized using Docker, providing exact parity between local developer environments, staging servers, and production clusters.[39] A highly optimized docker-compose.yml file orchestrates the microservices architecture, defining distinct, isolated containers for the React frontend (served efficiently via Nginx), the FastAPI backend application, the PostgreSQL relational database, the Redis message broker, and the Celery asynchronous worker nodes.[41] Furthermore, Judge0 requires its own specialized containerized cluster, leveraging Docker privileged modes to correctly establish the Linux execution namespaces required for secure sandboxing.[1]

Automated CI/CD via GitHub Actions

The deployment workflow is fully automated using GitHub Actions, enforcing strict quality

control and enabling zero-downtime updates in production.[38] The automated workflow is defined in a YAML configuration file (.github/workflows/ci_cd.yml) and executes the following deterministic sequence upon a pull request or merge to the main production branch:

1. **Code Quality and Linting:** The pipeline first executes static analysis tools, running Python flake8 and black on the backend repository, and ESLint on the React frontend to ensure strict code style adherence.[40]
2. **Automated Testing:** The runner spins up ephemeral, detached Docker containers to execute isolated pytest suites against the FastAPI endpoints and Jest tests for the frontend logic.[40] This ensures no regressions are introduced into the core evaluation or proctoring logic.
3. **Image Build and Push:** Upon the successful completion of all test suites, the GitHub Actions runner builds optimized, lightweight Docker images and pushes them to a secure container registry (such as DockerHub or AWS Elastic Container Registry) utilizing securely injected repository secrets.[44]
4. **Zero-Downtime Deployment:** The pipeline uses SSH protocols to access the production host environment (e.g., an AWS EC2 instance or Docker Swarm cluster). It pulls the latest container images and executes a rolling restart of the application services using Docker Compose. This strategy ensures that active user assessment sessions are not abruptly terminated during the deployment of new features or security patches.[44]

Feasibility Justification and Tool Selection Rationale

The selection of the technology stack is driven by rigorous technical analysis and architectural requirements, ensuring high feasibility for an enterprise-grade, MCA-level deployment project.

Technology Category	Selected Framework	Rejected Alternative	Technical Rationale for Best-Choice Selection
Backend Framework	**FastAPI**	Django / Node.js	Django's synchronous ORM and WSGI architecture cause severe thread-blocking delays during heavy concurrent load. Node.js is

			performant but lacks Python's native ecosystem for AI/ML pipeline integration. FastAPI perfectly merges high-speed asynchronous event loops (ASGI) with native Python machine learning libraries, achieving 2ms latencies.[2]
Database Architecture	**PostgreSQL**	MongoDB	While MongoDB excels at rapid unstructured data writes, SaaS multi-tenancy inherently requires strict relational data integrity and complex analytical dashboard aggregations. PostgreSQL's advanced JSONB capabilities and native Row-Level Security (RLS) make it vastly superior for enforcing mathematical tenant data isolation.[11]
Code Plagiarism Engine	**JPlag + MinHash**	MOSS / Basic String Matching	Simple string matching fails utterly against trivial variable renaming. MOSS

			operates effectively but relies on external servers and black-box algorithms. JPlag offers superior AST tokenization, and MinHash enables $O(N$ hashing scalability for massive datasets while operating entirely within local, secure infrastructure.[8]
Frontend UI/UX	**React + Monaco Editor**	Standard HTML Textareas	The Monaco editor provides standard IDE features (syntax highlighting, intelligent linting), significantly reducing cognitive friction and test-induced anxiety during coding evaluations compared to primitive text inputs.[1]

Phased Development Timeline

The implementation of the ProctoEase system is structured over a comprehensive six-month (24-week) development timeline, progressing sequentially from architectural discovery to final production deployment.[50] This phased approach ensures technical stability, security compliance, and continuous stakeholder alignment.

Phase 1: Architectural Discovery and Infrastructure Provisioning (Weeks 1–4)

The initial phase focuses on establishing the foundational data models and development environments.

- Finalize the relational database schemas and deploy the Row-Level Security (RLS) policies within PostgreSQL to guarantee multi-tenant isolation.
- Establish the baseline monorepo repository structure.
- Configure the primary Docker Compose environment for local development, encompassing FastAPI, PostgreSQL, and Redis.
- Implement the OAuth2 JWT-based authentication flows and formally define the Tenant-scoped Role-Based Access Control (RBAC) models.

Phase 2: Core Backend MVP and Assessment Execution Logic (Weeks 5–10)

This phase builds the core transactional logic required to facilitate examinations.

- Develop the CRUD REST APIs required for organizational onboarding, user management, and detailed examination configuration.
- Integrate the Celery worker nodes with the Redis message broker to handle background task orchestration.
- Deploy and configure the local Judge0 instance, establishing the secure network bridge between the FastAPI backend and the Linux sandboxes to safely execute incoming candidate code payloads.

Phase 3: Frontend Interface and Offline-First Engineering (Weeks 11–16)

The development focus shifts to the user experience and resilience mechanics.

- Develop the React SPA interfaces, including the analytical recruiter dashboards and the secure candidate assessment views.
- Integrate and configure the Monaco Editor component specifically for the coding assessment views.
- Implement the Service Worker caching strategies to ensure application shell persistence during network drops.
- Engineer the IndexedDB schema and the Background Synchronization API logic to guarantee offline resilience for examination submission payloads.

Phase 4: Edge AI Integration and Plagiarism Engines (Weeks 17–20)

Advanced algorithmic features are integrated during this highly technical phase.

- Embed the MediaPipe and face-api.js machine learning capabilities directly into the React client via WebAssembly for localized telemetry capture.
- Develop the backend heuristic risk-scoring aggregator to process and weigh incoming snapshot metadata.

- Build the Explainable AI (XAI) Interactive Evidence Timeline components for the administrative review dashboard.
- Implement the JPlag tokenization algorithms and the MinHash LSH clustering logic within the asynchronous Celery worker nodes.

Phase 5: Testing, DPDP Compliance Auditing, and Refinement (Weeks 21–22)

Rigorous quality assurance and legal compliance verification are conducted.

- Execute comprehensive end-to-end integration testing using modern frameworks like Playwright.
- Conduct load testing on the asynchronous backend to guarantee stability under high concurrent connection volumes simulating thousands of simultaneous test-takers.
- Audit the data pipelines to ensure absolute compliance with the DPDP Act 2023, specifically verifying data retention TTLs (Time to Live) and the Verifiable Parental Consent checkpoints.
- Refine user interface accessibility and optimize algorithmic performance based on testing feedback and edge-case discovery.

Phase 6: Production Deployment and CI/CD Automation (Weeks 23–24)

The system is transitioned from development environments to live production infrastructure.

- Configure the GitHub Actions pipeline to handle automated code testing and container image building.
- Provision the highly available cloud infrastructure (e.g., AWS EC2 instances, S3 storage buckets, managed RDS PostgreSQL).
- Implement zero-downtime deployment strategies via Docker Swarm or similar orchestration tools.
- Finalize comprehensive system documentation and transition the engineering team to ongoing monitoring, logging, and incident response protocols.

Works cited

1. Project Research and Analysis Canvas.pdf
2. FastAPI vs Django in 2025: Which is best for AI-Driven Web Apps? - Capsquery, accessed on February 12, 2026, https://capsquery.com/blog/fastapi-vs-django-in-2025-which-is-best-for-ai-driven-web-apps/
3. The Backend Framework Battle: FastAPI vs Node.js vs Django REST Framework - Medium, accessed on February 12, 2026, https://medium.com/@iamabdullah234/the-backend-framework-battle-fastapi-vs-node-js-vs-django-rest-framework-1420aecfd40c

4. High Availability and Horizontal Scaling with Celery - Six Feet Up, accessed on February 12, 2026, https://sixfeetup.com/blog/high-availability-scaling-with-celery
5. Scaling Agentic Workflows with Redis and Celery: Efficiently Managing Complexity in Modern Applications | by Savinu Aththanayake | Medium, accessed on February 12, 2026, https://medium.com/@nimetha.21/scaling-agentic-workflows-with-redis-and-celery-efficiently-managing-complexity-in-modern-450b3493fc23
6. The Accuracy of AI-Based Automatic Proctoring in Online Exams - ResearchGate, accessed on February 12, 2026, https://www.researchgate.net/publication/364345744_The_Accuracy_of_AI-Based_Automatic_Proctoring_in_Online_Exams
7. Learning Optimized Risk Scores, accessed on February 12, 2026, https://jmlr.org/papers/volume20/18-615/18-615.pdf
8. A Comparison of Three Popular Source code Similarity Tools for Detecting Student Plagiarism | Request PDF - ResearchGate, accessed on February 12, 2026, https://www.researchgate.net/publication/330156188_A_Comparison_of_Three_Popular_Source_code_Similarity_Tools_for_Detecting_Student_Plagiarism
9. jplag/JPlag: State-of-the-Art Source Code Plagiarism & Collusion Detection. Check for plagiarism in a set of programs. - GitHub, accessed on February 12, 2026, https://github.com/jplag/JPlag
10. Multi-tenant data isolation with PostgreSQL Row Level Security ..., accessed on February 12, 2026, https://aws.amazon.com/blogs/database/multi-tenant-data-isolation-with-postgresql-row-level-security/
11. Comparing MongoDB vs. PostgreSQL, accessed on February 12, 2026, https://www.mongodb.com/resources/compare/mongodb-postgresql
12. MongoDB vs PostgreSQL for SaaS Applications: Guide - Ciphernutz, accessed on February 12, 2026, https://ciphernutz.com/blog/mongodb-vs-postgresql-for-saas
13. PostgreSQL as NoSQL: The Complete Guide for SaaS Leaders - Medium, accessed on February 12, 2026, https://medium.com/@hashbyt/postgresql-as-nosql-the-complete-guide-for-saas-leaders-1c772b8ed107
14. Multi-tenant SaaS model with PostgreSQL, Mongo or DynamoDB? - Checkly, accessed on February 12, 2026, https://www.checklyhq.com/blog/building-a-multi-tenant-saas-data-model/
15. How to Secure Multi-Tenant Data with Row-Level Security in PostgreSQL - OneUptime, accessed on February 12, 2026, https://oneuptime.com/blog/post/2026-01-25-row-level-security-postgresql/view
16. Multi-Tenant Architecture with FastAPI: Design Patterns and Pitfalls | by Koushik Sathish, accessed on February 12, 2026, https://medium.com/@koushiksathish3/multi-tenant-architecture-with-fastapi-design-patterns-and-pitfalls-aa3f9e75bf8c
17. Using Service Workers - Web APIs | MDN, accessed on February 12, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Service_Worker_API/Using_Ser

vice_Workers
18. How to Build Offline-First React Apps Using IndexedDB & Service Workers? - Medium, accessed on February 12, 2026, https://medium.com/@sparklewebhelp/how-to-build-offline-first-react-apps-using-indexeddb-service-workers-b8380dbe86f5
19. Offline data - web.dev, accessed on February 12, 2026, https://web.dev/learn/pwa/offline-data
20. Background Synchronization API - Web APIs | MDN, accessed on February 12, 2026, https://developer.mozilla.org/en-US/docs/Web/API/Background_Synchronization_API
21. Build an Offline-First Mood Journal PWA with Next.js & IndexedDB - WellAlly, accessed on February 12, 2026, https://www.wellally.tech/blog/build-offline-first-pwa-nextjs-indexeddb
22. FastAPI JWT Authentication Tutorial – Secure Login & Role-Based Access Control, accessed on February 12, 2026, https://prateekmondal.great-site.net/blog/fastapi-jwt-authentication-tutorial-secure-login-role-based-access-control/
23. Building Multi-Tenant APIs with FastAPI and Subdomain Routing: A Complete Guide, accessed on February 12, 2026, https://medium.com/@diwasb54/building-multi-tenant-apis-with-fastapi-and-subdomain-routing-a-complete-guide-cc076cb02513
24. Protect your FastAPI with RBAC and JWT validation - Logto docs, accessed on February 12, 2026, https://docs.logto.io/api-protection/python/fastapi
25. Multi-tenant Role-based Access Control (RBAC) - Aserto, accessed on February 12, 2026, https://www.aserto.com/use-cases/multi-tenant-saas-rbac
26. How to Choose the Right Authorization Model for Your Multi-Tenant SaaS Application, accessed on February 12, 2026, https://auth0.com/blog/how-to-choose-the-right-authorization-model-for-your-multi-tenant-saas-application/
27. Building Role-Based Access Control for a Multi-Tenant SaaS Startup - Medium, accessed on February 12, 2026, https://medium.com/@my_journey_to_be_an_architect/building-role-based-access-control-for-a-multi-tenant-saas-startup-26b89d603fdb
28. Best user management service with FastAPI? - Reddit, accessed on February 12, 2026, https://www.reddit.com/r/FastAPI/comments/1kwg55d/best_user_management_service_with_fastapi/
29. India's Digital Personal Data Protection Act 2023 brought into force - Hogan Lovells, accessed on February 12, 2026, https://www.hoganlovells.com/en/publications/indias-digital-personal-data-protection-act-2023-brought-into-force-
30. Teaching in the Age of DPDPA: When Child Safety Becomes EdTech's Competitive Advantage - Tsaaro Consulting, accessed on February 12, 2026, https://tsaaro.com/blogs/teaching-in-the-age-of-dpdpa-when-child-safety-beco

mes-edtechs-competitive-advantage/
31. Understanding Data Privacy and DPDP Act - Seqrite, accessed on February 12, 2026, https://www.seqrite.com/understanding-data-privacy-and-dpdp-act/
32. Children's Data Protection Under India's DPDP Rules, accessed on February 12, 2026, https://ksandk.com/data-protection-and-data-privacy/childrens-data-protection-under-indias-dpdp-rules/
33. Data Dignity in Education What DPDP 2023 Means for Schools Universities, accessed on February 12, 2026, https://dpdpaforschools.in/public/blogs/data-dignity-in-education-what-dpdp-2023-means-for-schools-universities?page=2
34. Protecting Minor's Data in India: DPDP, EdTech Privacy Compliance & Practical Checklist, accessed on February 12, 2026, https://www.idfy.com/blog/protecting-minors-data-in-india-dpdp-edtech-privacy-compliance-practical-checklist/
35. DPDP 2023/2025: Retention & Erasure Made Simple - Pricoris, accessed on February 12, 2026, https://pricoris.com/blog/dpdp-retention-erasure-guide/
36. Navigating Privacy Risks For Edtech Platforms And Educational Institutions Under India's Data Protection Regime: Children's Data, Consent And Compliance - King Stubb & Kasiva, accessed on February 12, 2026, https://ksandk.com/data-protection-and-data-privacy/dpdp-act-compliance-for-edtech-schools/
37. Digital Personal Data Protection Compliance Checklist - DPO India, accessed on February 12, 2026, https://dpo-india.com/Blogs/dpdp-compliance/
38. Actions · alperencubuk/fastapi-celery-redis-postgres-docker-rest-api - GitHub, accessed on February 12, 2026, https://github.com/alperencubuk/FastAPI-Celery-Redis-Postgres-Docker-REST-API/actions
39. Dockerizing Celery and FastAPI - TestDriven.io, accessed on February 12, 2026, https://testdriven.io/courses/fastapi-celery/docker/
40. What is the right way to deploy a FastAPI app? : r/learnpython - Reddit, accessed on February 12, 2026, https://www.reddit.com/r/learnpython/comments/1d45gl6/what_is_the_right_way_to_deploy_a_fastapi_app/
41. FastAPI-PostgreSQL-Celery-RabbitMQ-Redis backend with Docker containerization - Reddit, accessed on February 12, 2026, https://www.reddit.com/r/FastAPI/comments/nshn5b/fastapipostgresqlceleryrabbitmqredis_backend_with/
42. docker-compose.yml - evert0n/sample-fastapi-react - GitHub, accessed on February 12, 2026, https://github.com/evert0n/sample-fastapi-react/blob/main/docker-compose.yml
43. How to Set Up a FastAPI + PostgreSQL + Celery Stack with Docker Compose - OneUptime, accessed on February 12, 2026, https://oneuptime.com/blog/post/2026-02-08-how-to-set-up-a-fastapi-postgresql-celery-stack-with-docker-compose/view

44. CICD Pipeline from Scratch using GitHub Actions & Docker Swarm - YouTube, accessed on February 12, 2026, https://www.youtube.com/watch?v=oATILGcBEYQ
45. Deploy a FastAPI application using Docker and GitHub Actions for CI/CD(Part Three) | by Adebisi Olayinka | Medium, accessed on February 12, 2026, https://medium.com/@adebisiolayinka30/deploy-a-fastapi-application-using-github-actions-for-ci-cd-07f0f44e549e
46. Full stack, modern web application template. Using FastAPI, React, SQLModel, PostgreSQL, Docker, GitHub Actions, automatic HTTPS and more., accessed on February 12, 2026, https://github.com/fastapi/full-stack-fastapi-template
47. Deploying a FastAPI Application with CI/CD Pipeline | by Ktrontech | Medium, accessed on February 12, 2026, https://medium.com/@williamtijesuni/deploying-a-fastapi-application-with-ci-cd-pipeline-7641f6ce7c16
48. Fastapi vs Django vs flask vs node : r/developersPak - Reddit, accessed on February 12, 2026, https://www.reddit.com/r/developersPak/comments/1jz74ak/fastapi_vs_django_vs_flask_vs_node/
49. Codequiry vs Dolos vs MOSS vs JPlag — Complete 2024 Code Plagiarism Detection Comparison, accessed on February 12, 2026, https://codequiry.com/codequiry-vs-dolos-vs-moss
50. AI Development Timeline: A Startup Founder's Guide - Zackriya Solutions, accessed on February 12, 2026, https://www.zackriya.com/ai-development-timeline-honest-guide-for-startup-founders/
51. AI Developer Roadmap For Full-Stack Devs | PDF | Machine Learning - Scribd, accessed on February 12, 2026, https://www.scribd.com/document/959156832/AI-Developer-Roadmap-for-Full-Stack-Devs-1